

Factory Patterns e Dependency Injection em .NET

Material Educacional Completo para Graduação em Computação



Sobre Este Material

Este é um pacote educacional completo e integrado desenvolvido especificamente para o ensino de padrões Factory (Simple Factory, Factory Method, Abstract Factory e Project Factory) em conjunto com Dependency Injection no ecossistema .NET moderno. O material foi estruturado para alunos de nível intermediário a avançado em Ciência da Computação que já possuem conhecimento básico de C#, orientação a objetos, interfaces e generics.

O diferencial desta abordagem pedagógica é a integração profunda entre os conceitos de Dependency Injection e Factory Patterns, demonstrando como esses padrões trabalham juntos em aplicações empresariais reais do .NET. Em vez de ensinar os padrões de forma isolada e acadêmica, este material mostra como eles são aplicados no contexto profissional de desenvolvimento de software.



Objetivos de Aprendizado

Ao completar este material, os alunos serão capazes de compreender profundamente o princípio de Inversão de Controle e como Dependency Injection implementa este princípio no .NET, dominar o uso do container de DI Microsoft.Extensions.DependencyInjection incluindo configuração de lifetimes e resolução de dependências, identificar quando cada padrão Factory é apropriado e justificar tecnicamente suas escolhas, implementar factories que se integram corretamente com DI seguindo melhores práticas, e desenvolver sistemas modulares extensíveis usando Project Factory para orquestração de módulos.

Além disso, os alunos serão capazes de escrever testes unitários e de integração efetivos para factories, aplicar princípios SOLID em suas implementações de forma prática e não apenas teórica, avaliar trade-offs entre diferentes padrões considerando extensibilidade, performance e complexidade, e evitar anti-patterns comuns como Service Locator e God Factory.



Estrutura do Material

O material está organizado em cinco documentos principais que formam uma progressão didática coerente, começando pela teoria fundamental, passando por exemplos práticos de código, avançando para diagramas visuais de arquitetura, chegando aos exercícios práticos com avaliação, e finalizando com material de suporte para produtividade dos alunos.

Documento 01: TEORIA-COMPLETA.md (Fundamentos Teóricos - 20 páginas) apresenta oito módulos progressivos que cobrem desde os conceitos fundamentais de Dependency Injection até implementações avançadas de Project Factory. O primeiro módulo explica o problema do acoplamento forte e como DI resolve este problema através de Inversão de Controle. O segundo módulo detalha o uso de Microsoft.Extensions.DependencyInjection incluindo IServiceCollection, IServiceProvider e gerenciamento de lifetimes. O terceiro módulo introduz os padrões Factory clássicos começando com Simple Factory e evoluindo para Factory Method e Abstract Factory. O quarto módulo demonstra a integração entre factories e DI mostrando como usar IServiceProvider, padrões com Func<T> e generics, e implementações de Named Factory.

O quinto módulo apresenta o conceito de Project Factory para orquestração de módulos em sistemas extensíveis. Os módulos seguintes cobrem boas práticas e anti-patterns, estratégias de teste completas, casos reais com análise de trade-offs, e materiais de referência rápida.

Documento 02: EXEMPLOS-CODIGO-COMPLETO.md (Implementações Práticas) contém cinco projetos .NET completos e executáveis que demonstram progressivamente os padrões estudados. O primeiro exemplo implementa Simple Factory básico e sua evolução para integração com DI, demonstrando o problema do acoplamento e sua solução. O segundo exemplo mostra Factory Method com template method pattern usando o domínio de notificações multi-canal. O terceiro exemplo apresenta Abstract Factory completo para geração de relatórios em múltiplos formatos, garantindo consistência entre famílias de produtos. O quarto exemplo demonstra padrões avançados de integração com DI incluindo Func<T> factories, generic factories e named factories usando o domínio de processamento de pagamentos. O quinto e mais complexo exemplo implementa um sistema modular completo com Project Factory, descoberta dinâmica de módulos, resolução de dependências entre módulos e orquestração de inicialização. Todos os exemplos incluem código completo, comentado em português, pronto para executar com dotnet CLI, e demonstrando integração adequada com DI.

Documento 03: DIAGRAMAS-UML.md (Visualizações de Arquitetura) fornece doze diagramas técnicos em código fonte PlantUML e Mermaid que podem ser renderizados imediatamente. Os diagramas incluem class diagrams para cada padrão Factory mostrando interfaces, implementações e relações, sequence diagrams demonstrando fluxo de criação de objetos e interação com DI container, diagramas de arquitetura em Mermaid mostrando onde factories se encaixam no sistema, diagramas de comparação entre diferentes padrões Factory, e árvores de decisão para escolha do padrão apropriado baseado em requisitos. Todos os diagramas vêm com código fonte versionável que pode ser renderizado online ou em extensões de IDE, são exportáveis para PNG e SVG de alta qualidade para uso em apresentações, e incluem anotações explicativas sobre decisões de design.

Documento 04: EXERCICIOS-E-TESTES.md (Prática e Avaliação) apresenta três exercícios progressivos com complexidade crescente, templates reutilizáveis de testes unitários e de integração, checklist detalhado para code review de factories, comparações lado a lado de código incorreto versus correto, e rubricas completas de avaliação com critérios objetivos. O exercício básico sobre sistema de logging requer implementação de Simple e Factory Method, levando duas a três horas com foco em fundamentos e testes básicos. O exercício intermediário sobre sistema de exportação requer implementação de Abstract Factory completo, levando quatro a cinco horas com foco em famílias de produtos e integração DI. O exercício avançado sobre sistema modular de e-commerce requer implementação de Project Factory com múltiplos módulos interdependentes, levando oito a dez horas com requisitos de arquitetura modular, documentação completa e apresentação oral. Cada exercício inclui descrição detalhada de requisitos, critérios objetivos de avaliação com pontuação, rubricas para diferentes níveis de desempenho, e templates de testes prontos para adaptação.

Documento 05: PROMPTS-E-APRESENTACAO.md (Produtividade e Comunicação) oferece oito prompts otimizados para uso com ferramentas de IA que permitem gerar esqueleto de código de factories, criar suítes de testes unitários automaticamente, gerar diagramas UML a partir de código, produzir documentação README profissional, criar estrutura de slides para apresentação, refatorar código legacy para usar factories, realizar code review automatizado, e identificar casos de teste faltantes. O documento também inclui guia detalhado para apresentação oral cobrindo estrutura recomendada de dez a quinze minutos, técnicas de preparação e ensaio, dicas para slides efetivos e legíveis, estratégias para gerenciar nervosismo e tempo, como responder perguntas com confiança, e checklist completo de preparação pré-apresentação. Adicionalmente,

oferece framework de autoavaliação pós-apresentação, links para ferramentas úteis para diagramação e apresentação, e recomendações de leituras complementares.

Como Usar Este Material

Para Professores, este material oferece uma sequência didática testada e completa. Recomenda-se seguir a ordem dos módulos no documento de teoria para construir conhecimento progressivamente, usar os exemplos de código como demonstrações em aula executando-os ao vivo para mostrar os conceitos em ação, projetar os diagramas durante explicações teóricas para visualização dos conceitos, atribuir exercícios progressivos começando pelo básico e avançando conforme a turma demonstra domínio, e usar as rubricas fornecidas para avaliação objetiva e justa. A carga horária sugerida é de oito horas-aula distribuídas em quatro encontros de duas horas cada, com tempo adicional para os alunos completarem exercícios práticos. Este cronograma pode ser ajustado conforme a necessidade e o ritmo da turma.

Para Alunos, este material serve como guia completo de estudo autodirigido. Inicie lendo o documento de teoria do início ao fim para compreensão conceitual ampla. Em seguida, execute cada exemplo de código localmente modificando-os para experimentar e consolidar o aprendizado. Estude os diagramas UML para visualizar as relações entre componentes e o fluxo de execução. Complete os exercícios práticos em ordem crescente de dificuldade, e use os prompts fornecidos para acelerar desenvolvimento e gerar material auxiliar. Ao preparar apresentações, siga o guia detalhado de estrutura e dicas fornecidas. Lembre-se que a melhor forma de aprender programação é praticando, então dedique tempo significativo aos exercícios e experimente modificar os exemplos para testar sua compreensão.

Requisitos Técnicos

Para trabalhar com este material, você precisará ter instalado o .NET SDK 8.0 ou superior que pode ser baixado gratuitamente em dotnet.microsoft.com/download. Uma IDE é recomendada, sendo as opções Visual Studio 2022 Community que oferece experiência completa com ferramentas visuais, Visual Studio Code com extensões C# e PlantUML que é leve e multiplataforma, ou JetBrains Rider que é pago mas extremamente poderoso. Para execução e testes dos exemplos, você precisará de conhecimento básico de comandos dotnet CLI para compilar e executar projetos. Para visualização de diagramas, acesso às ferramentas online PlantUML em plantuml.com e Mermaid Live em mermaid.live é suficiente. Para testes, familiaridade básica com xUnit framework que é o padrão usado em todos os exemplos é necessária.

Embora não sejam obrigatórios, alguns conhecimentos opcionais são recomendados para melhor aproveitamento do material. Familiaridade com Git para versionamento e entrega de exercícios facilita a organização do código. Conhecimento de padrões SOLID ajuda a entender as decisões de design apresentadas. Experiência com Moq ou outro framework de mocking é útil para testes mais avançados. Compreensão básica de UML facilita a leitura dos diagramas de classe e sequência.

Tópicos Cobertos em Detalhes

Dependency Injection é coberto de forma abrangente incluindo o conceito fundamental de Inversão de Controle e como ele muda o paradigma de desenvolvimento. O documento explica em profundidade `Microsoft.Extensions.DependencyInjection` com `IServiceCollection` para registro e `IServiceProvider` para resolução. Os três tipos de lifetime (`Transient`, `Scoped` e `Singleton`) são explicados com exemplos práticos de

quando usar cada um e as consequências de escolhas incorretas. Constructor Injection é apresentado como a técnica preferencial com comparação clara contra Property e Method Injection. O documento ensina como integrar DI com Generic Host para aplicações console e serviços. Especial atenção é dada para identificar e evitar o anti-pattern Service Locator que é um erro comum ao aprender DI. Finalmente, técnicas de teste com DI são demonstradas mostrando como criar mocks e stubs de dependências.

Factory Patterns são apresentados em progressão lógica começando com Simple Factory explicando quando é apropriado seu uso e suas limitações claras, servindo como ponte para padrões mais sofisticados. Factory Method é ensinado com foco em como ele promove extensibilidade via herança e sua integração natural com Template Method pattern. Abstract Factory é detalhado mostrando como criar famílias consistentes de objetos relacionados e garantir compatibilidade entre produtos. O conceito de Project Factory é introduzido como padrão arquitetural para sistemas modulares, incluindo descoberta dinâmica de módulos, resolução automática de dependências entre módulos, e orquestração de registro e inicialização. Cada padrão é demonstrado com três domínios práticos: notificações multi-canal, relatórios em múltiplos formatos, e processamento de pagamentos, garantindo que os alunos vejam aplicações variadas dos conceitos.

Integração entre Factories e DI é um tópico central deste material que o diferencia de abordagens tradicionais. O documento explica como factories devem usar IServiceProvider para resolver dependências em vez de instanciação direta. Demonstra o uso de Func<T> para criar factories funcionais inline que são idiomáticas em C#. Apresenta implementação de Generic Factories com type-safe resolution que aumentam a segurança de tipos. Mostra Named Factories com dictionary-based selection para cenários onde você precisa selecionar entre múltiplas implementações baseado em chave. Explica configuração adequada de lifetimes para factories e seus produtos, incluindo as regras críticas de segurança sobre captive dependencies. Demonstra como testar factories integradas com DI usando service providers de teste.

Princípios SOLID em Factories são aplicados de forma prática através de exemplos concretos. Single Responsibility é demonstrado mostrando factories focadas apenas em criação sem lógica de negócio. Open/Closed é ilustrado com factories extensíveis que não precisam ser modificadas para adicionar novos tipos. Liskov Substitution é aplicado garantindo que todas as implementações de factory interface sejam verdadeiramente intercambiáveis. Interface Segregation é mostrado através de interfaces coesas e específicas em vez de monolíticas. Dependency Inversion é o princípio central com todos os componentes dependendo de abstrações e não de concreções.

Boas Práticas Enfatizadas

Este material enfatiza consistentemente várias boas práticas essenciais para desenvolvimento profissional. Factories devem sempre trabalhar com interfaces ao invés de classes concretas para baixo acoplamento. Constructor Injection é preferível a outras formas de injeção por tornar dependências explícitas e obrigatórias. Validação rigorosa de entrada com exceções específicas e mensagens informativas previne erros em runtime. Nunca retornar null de factories evita null reference exceptions no código cliente. Factories devem ser stateless ou garantir thread-safety se mantiverem estado. Código deve estar bem documentado especialmente tipos suportados e exceções possíveis. Testes devem cobrir tanto casos de sucesso quanto casos de erro e edge cases. Lifetimes do DI container devem ser escolhidos cuidadosamente baseado em requisitos de estado e performance. Cada componente deve ter responsabilidade única seguindo SRP rigorosamente.

Anti-Patterns a Evitar

O material identifica e explica claramente vários anti-patterns comuns que devem ser evitados. God Factory que conhece e cria muitos tipos diferentes violando SRP deve ser quebrado em múltiplas factories especializadas. Service Locator que injeta IServiceProviders desnecessariamente deve ser substituído por injeção direta de dependências necessárias. Factories com estado mutável compartilhado que não são thread-safe devem ser refatorados para serem stateless. Factories que retornam null forçando verificações em todo código cliente devem lançar exceções específicas. Acoplamento forte com instanciação direta via new deve ser substituído por resolução via DI container. Lógica de negócio dentro de factories que deveria estar em serviços especializados deve ser extraída. Factory Methods excessivamente complexos com muitos níveis de condicionais devem usar padrões de configuração mais claros.

Exercícios Práticos

Os três exercícios progressivos oferecem experiência prática crescente. O Exercício Básico Sistema de Logging implementa Simple Factory e Factory Method, requer duas a três horas de trabalho, vale cem pontos com critérios objetivos, e foca em fundamentos de factories e testes básicos. O Exercício Intermediário Sistema de Exportação implementa Abstract Factory completo, requer quatro a cinco horas de trabalho, vale cem pontos com foco em famílias de produtos, e enfatiza integração DI e testes abrangentes. O Exercício Avançado Sistema Modular de E-commerce implementa Project Factory com módulos interdependentes, requer oito a dez horas de trabalho, vale cem pontos com múltiplos critérios, e exige arquitetura modular completa, documentação profissional e apresentação oral.

Cada exercício inclui requisitos detalhados e não ambíguos, critérios de avaliação objetivos com pontuação clara, rubricas distinguindo excelente, bom, satisfatório e insatisfatório, templates de testes prontos para adaptação, e exemplos de entregáveis esperados. Os exercícios cobrem três domínios distintos - logging, exportação de dados e e-commerce - para demonstrar versatilidade dos padrões.

Avaliação e Feedback

O sistema de avaliação foi desenhado para ser justo, objetivo e formativo. Rubricas detalhadas fornecem critérios claros para cada nível de desempenho eliminando subjetividade. Checklist de code review permite verificação sistemática de qualidade. Templates de teste garantem padrões mínimos de verificação. Comparações lado a lado errado versus correto demonstram claramente conceitos. Feedback deve ser específico citando trechos de código e sugerindo melhorias concretas. Avaliação deve considerar não apenas se funciona mas qualidade de design, testabilidade e extensibilidade. Apresentação oral avalia não só conhecimento técnico mas habilidade de comunicação.

Solução de Problemas Comuns

Alguns problemas são comumente encontrados por alunos ao trabalhar com este material. Se os exemplos de código não compilam, verifique se você tem .NET 8 SDK instalado com dotnet --version, confirme que todas as dependências foram restauradas com dotnet restore, e verifique se está no diretório correto com o arquivo .csproj.

Se testes estão falhando, certifique-se que construiu o projeto antes com dotnet build, execute testes de um por vez para isolar problemas, verifique se mocks estão configurados corretamente, e leia mensagens de erro cuidadosamente pois elas geralmente indicam o problema exato.

Se diagramas UML não renderizam, verifique se copiou o código completo incluindo `@startuml` e `@enduml`, confirme que está usando um renderizador PlantUML compatível, e teste com exemplos simples primeiro para validar sua configuração.

Se integração DI apresenta problemas, confirme que todos os serviços necessários foram registrados, verifique se os lifetimes são apropriados para seu caso, nunca injete `Transient` ou `Scoped` em `Singleton`, e use `GetRequiredService` em vez de `GetService` para falhar rapidamente se serviço não está registrado.



Recursos Adicionais

Para continuar aprendendo após este material, várias fontes são recomendadas. A documentação oficial Microsoft sobre Dependency Injection e Generic Host em docs.microsoft.com é sempre atualizada e autoritativa. O livro *Dependency Injection Principles, Practices, and Patterns* de Steven van Deursen e Mark Seemann é o texto definitivo sobre DI. O clássico *Design Patterns: Elements of Reusable Object-Oriented Software* do Gang of Four apresenta os padrões originais. *Clean Architecture* de Robert C. Martin oferece contexto arquitetural mais amplo. O blog de Mark Seemann em blog.ploeh.dk contém artigos aprofundados sobre DI e padrões. A comunidade .NET Foundation oferece exemplos de código aberto demonstrando estes padrões em projetos reais.



Contribuindo e Feedback

Este material foi desenvolvido para ser um recurso vivo que melhora com uso e feedback. Se você encontrou erros ou tem sugestões de melhoria, seu feedback é valioso. Se você é instrutor e usou este material, compartilhe sua experiência e o que funcionou ou não com sua turma. Se você é aluno, relate quais partes foram mais difíceis de entender para que possam ser melhoradas. Contribuições de novos exemplos de código, exercícios adicionais ou explicações alternativas são bem-vindas.



Licença e Uso

Este material foi preparado especificamente para fins educacionais e acadêmicos. Professores são encorajados a adaptar e estender o material para suas necessidades específicas de curso. Alunos podem usar este material para estudo pessoal e desenvolvimento de portfólio. Organizações podem usar para treinamento interno de equipes. Recomenda-se atribuição adequada ao usar ou adaptar este material.



Próximos Passos

Após dominar este material, os próximos passos recomendados para continuar sua jornada de aprendizado incluem estudar outros padrões criacionais como `Builder`, `Prototype` e `Singleton` em profundidade. Explorar padrões estruturais como `Adapter`, `Decorator` e `Composite` para complementar seu conhecimento. Investigar padrões comportamentais como `Strategy`, `Observer` e `Command` que frequentemente trabalham com `factories`. Aprender sobre arquiteturas como *Clean Architecture*, *Onion Architecture* e *Vertical Slice Architecture*. Praticar desenvolvimento orientado a testes aumentando progressivamente a cobertura e qualidade dos testes. Contribuir para projetos open source que usam estes padrões para ganhar experiência real. Aplicar estes conceitos em projetos pessoais para consolidar conhecimento através da prática.

Informações de Suporte

Este material foi desenvolvido como um recurso educacional completo e autocontido. Para perguntas sobre conceitos específicos, consulte primeiro a seção correspondente no documento teórico que geralmente contém exemplos detalhados. Para problemas técnicos com exemplos de código, verifique a seção de solução de problemas comuns neste README. Para discussões sobre exercícios e avaliações, consulte as rubricas e critérios fornecidos que são intencionalmente detalhados. Para sugestões de melhoria ou feedback sobre o material, contribuições construtivas são sempre bem-vindas.

Versão do Material: 1.0

Data de Criação: Dezembro 2024

Framework Alvo: .NET 8 (LTS)

Nível: Intermediário/Avançado

Carga Horária Sugerida: 8 horas-aula + exercícios práticos

Desenvolvido com foco em: Qualidade técnica, progressão didática clara, exemplos práticos e executáveis, integração profunda entre conceitos, preparação para mercado profissional.